

RiskMap

Safest-route navigation for New York City · build plan v3

What we want: a Next.js map app where you type where you are and where you're going. Instead of the fastest route, it returns the **safest** one — steering around streets with a history of injury crashes, around violent-crime clusters, and around known street-flooding locations when the weather API says heavy rain is hitting that borough. Two routes are drawn side by side so the difference is obvious.

1 · Stack

LAYER	CHOICE	WHY THIS ONE
Frontend	Next.js (App Router)	App + API routes in one deploy on Vercel
Database	Supabase Postgres + PostGIS	Enable the PostGIS extension — real distance queries and "snap point to nearest road" in SQL instead of Python
Map	Mapbox GL JS	50k loads/month free, no billing card. Includes geocoding (address → lat/lon) so we don't need a second API. Google Maps requires a billing account attached.
Weather	Open-Meteo	No API key, no signup. 5 calls, one per borough.
Routing	A* in a Next.js API route	Graph loaded from Supabase, cached in memory
Prep	Python notebooks → Supabase	One-off. Never runs during the demo.

2 · The three data layers

LAYER	FILE	SIZE	STATE
Crashes	Motor_Vehicle_Collisions...csv	2,269,187 rows	READY
Flooding	FloodNet__Street_Flooding...csv	2,929 events	NEEDS GEOCODING
Crime	sample_complaints.csv	20,000 rows	SCORED · INPUT SYNTHETIC

Crashes — the strong layer

2.27M CRASHES 2012–2026	756,353 INJURED	3,617 KILLED
----------------------------	--------------------	-----------------

Clean `LATITUDE / LONGITUDE` on 89% of rows plus injury and fatality counts — everything the risk score needs. **Use 2022 onward only** (413,780 rows): crash volume fell from ~230k/year pre-COVID to ~86k in 2025, so older data describes traffic that no longer exists.

Two traps: 7,591 rows sit at exactly `(0, 0)` — filter by NYC bounding box, not just `NOT NULL`. And 30% of rows have no `BOROUGH`, so filter by coordinates instead.

Flooding — real, but no coordinates

2,929 measured flood events from 294 FloodNet sensors, Nov 2020 → Aug 2026, and **accelerating**: 142 events in 2022 versus 1,021 already in 2026. Median peak depth 3.5 inches; 1,295 events reached 4 inches or more.

The file has no latitude or longitude column. Location lives only inside `Sensor Name` , as a borough prefix plus a street: `Q - Beach 84 St` , `BX - Ditmars St/Hunter Ave 2` . Fix: strip the prefix and trailing number, geocode the 294 unique names once with Mapbox, store in Supabase. One-time script, not a runtime cost.

Flooding is highly concentrated, which helps: Queens alone accounts for 1,710 of 2,929 events (mostly the Rockaways), then Bronx 485, Brooklyn 425, Staten Island 223, Manhattan 79. `Q - Beach 84 St` alone logged 438 floods. A small number of avoid-zones covers most of the real risk.

Crime — scored and cleaned

Pipeline built and verified: `classify_complaints.ipynb` . Severity is ranked by danger to someone *walking past*, which is what the router cares about.

#	MEANING	MATCHED OFFENSES
5	Force or threat against a person	murder, rape, shooting, felony assault, robbery
4	Violence or weapons, lesser degree	weapons, assault 3 , harassment, arson
3	Intrusion into occupied space	burglary , trespass
2	Property loss or damage	grand larceny, criminal mischief
1	Petty property	petit larceny , fraud, forgery

Rules are keyword-based rather than exact-string, so they survive this file's truncated labels (`CRIMINAL MISCHIEF & RELATED OF`) **and** already cover the full NYPD taxonomy — swapping in the real file needs no code changes. Order matters: `FELONY ASSAULT` is tested before plain `ASSAULT` . An audit cell surfaces anything falling through to the default rather than letting it be silently scored 1.

20,000 rows → 14,326 on-street → 4,823 at severity ≥ 4

But the input file is fabricated

The coordinates are *not* pure noise — they cluster (variance/mean = 479, where random would be 1.0). But they don't sit on the city. Fraction of points within 50 m of a real NYC street:

Real crash locations (<i>ceiling</i>)	100.0%
<code>sample_complaints.csv</code>	62.6%
Uniform random in the same box (<i>floor</i>)	45.1%

Closer to the random floor than to real data. Roughly **1 point in 3 is somewhere a crime cannot have happened** — the 90th percentile is 1.5 km from the nearest street, the maximum 8.1 km, which is open water.

The labels are separately broken:

- Stated borough vs. borough-by-coordinate agrees **exactly 25.0%** — chance for a 4-way guess. `BORO_NM` is shuffled relative to the coordinates.
- All 20,000 rows share the timestamp `12:00:00` — no time-of-day signal exists.
- Only 6 offense types, each ~3,300 rows — filtering to assault returns a random quarter of the same cloud.

What to do: download the real *NYPD Complaint Data Historic* from NYC Open Data — identical column names, so the notebook runs unchanged and produces a genuine layer. If there's no time, ship crashes + flooding and drop crime. Two real layers beat three where one is fabricated.

3 · How risk becomes a route

Every hazard collapses to one number per street segment. No model, no training — a weighted sum tuned by eye.

$$\text{cost} = \text{length} \times (1 + \alpha \cdot \text{crash_risk} + \beta \cdot \text{flood_risk} \cdot \text{raining} + \gamma \cdot \text{crime_risk})$$

TERM	RANGE	SOURCE
<code>crash_risk</code>	0 – 1	Crashes within 50 m, weighted <code>1 + 3·injuries + 10·deaths</code> , percentile-ranked
<code>flood_risk</code>	0 – 1	Segment within 200 m of a FloodNet sensor, scaled by that sensor's event count
<code>crime_risk</code>	0 – 1	Scored complaints within 100 m, weighted by severity, percentile-ranked
<code>raining</code>	0 or 1	Open-Meteo says heavy rain in <i>that segment's borough</i>
<code>α, β, γ</code>	~2 – 5	Hand-tuned sliders. $\alpha=3$ makes a max-risk street feel 4× longer.

Keep the penalty multiplicative. Risk only ever pushes cost *upward*, so $\text{cost} \geq \text{length}$ always holds — which keeps straight-line distance a valid A* heuristic and the routes correct. If safe streets were ever made *cheaper* than their true length, A* would return wrong paths and nothing would visibly break.

4 · Weather, per borough

Open-Meteo, five calls — one per borough centroid. No key required.

```
const BOROS = {
  manhattan: [40.7831, -73.9712],
  brooklyn: [40.6782, -73.9442],
  queens: [40.7282, -73.7949],
  bronx: [40.8448, -73.8648],
  staten_island: [40.5795, -74.1502],
};

GET api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&hourly=precipitation

raining[boro] = max(precipitation, next 3h) > 4mm/h
```

Cache in Supabase for 15 minutes so repeated routing requests don't re-hit the API. Five booleans is the entire weather payload.

Per-borough is the right resolution, and it's honest. Rain genuinely varies across a 50 km city, so five readings beat one. But it does not vary block to block — the *geography* of flood risk comes from the FloodNet sensors, not the weather API. If a judge asks how granular the weather is: "five boroughs, and the flood sensors supply the street-level detail."

5 · Supabase schema

```
create extension if not exists postgis;

create table road_edges (
  id bigserial primary key,
  from_node bigint,
  to_node bigint,
  geom geography(LineString, 4326),
  length_m real,
  borough text,
  crash_risk real, -- 0-1
  flood_risk real, -- 0-1
  crime_risk real -- 0-1
);
create index on road_edges using gist (geom);

create table flood_sensors (
  sensor_name text primary key,
  borough text,
  geom geography(Point, 4326),
  event_count int,
  max_depth_in real
);

create table crime_points (
  id bigserial primary key,
  occurred_on date,
  offense text,
  severity smallint, -- 1-5
  bucket text, -- violent | intrusion | property
  weight real,
  geom geography(Point, 4326)
);
create index on crime_points using gist (geom);

create table weather_cache (
  -- 5 rows, refreshed every 15 min
```

```

    borough    text primary key,
    raining    bool,
    checked_at timestampz
  );

```

The three `_risk` columns are written once by the prep notebooks. At request time the API route reads `road_edges`, applies the weather booleans, and runs A*. **Nothing heavy happens during the demo.**

6 · Build order

#	PHASE	TIME	WHAT
1	Map + graph	~1 hr	Next.js page with Mapbox, two address inputs via Mapbox geocoding. Load the Manhattan street graph (OSMnx → <code>road_edges</code>), run plain A* on distance. <i>No risk data yet</i> — but already a working routing app, so we can never end with nothing.
2	Crash layer	~2 hrs	Filter 2022+, drop the <code>(0,0)</code> rows, snap to nearest edge with PostGIS, aggregate the injury-weighted score, percentile-rank, write <code>crash_risk</code> . Test on 10k before all 414k.
3	Two-route comparison	~30 min	Run A* twice per request — once on length, once on risk-weighted cost. Draw grey and green lines. This is where it becomes a demo instead of a map.
4	Crime layer	~30 min	Load <code>complaints_scored.csv</code> into <code>crime_points</code> , set <code>crime_risk</code> on edges within 100 m. <i>Notebook already done.</i>
5	Flood layer	~1 hr	Geocode the 294 sensor names, load <code>flood_sensors</code> , set <code>flood_risk</code> on edges within 200 m, add the β term behind a manual rain toggle.
6	Weather API	~20 min	Replace the toggle with 5 live Open-Meteo calls. Keep the manual override — it probably won't be raining during the demo.
7	Polish	what's left	Reason string ("avoided 3 high-crash intersections, +4 min"), heat-map toggles, $\alpha/\beta/\gamma$ sliders judges can drag.

7 · Demo

1. Enter start and destination. Grey line appears — "this is what Google gives you."
2. Hit *Safe route*. Green line diverges around the high-crash corridors. "+4 minutes, avoids three intersections with 2,000 crashes on record."
3. Flip heavy rain on for Queens. The green route **moves again**, now dodging the Rockaways flood cluster.
4. Drag the α slider up and watch the route grow more cautious in real time.

Protect one thing: finish step 3 early. Crashes + two-route comparison is already a complete, defensible project. Crime, flood and weather make it better — they are not prerequisites for having something that works.

8 · Repo

FILE	WHAT
<code>inspect_dataset.ipynb</code>	Generic dataset profiler — set <code>DATA_PATH</code> , run all
<code>classify_complaints.ipynb</code>	Crime severity scoring 1–5 + geographic cleaning → <code>complaints_scored.csv</code>
<code>RiskMap_Plan.md</code> / <code>.pdf</code>	This plan
<code>.gitignore</code>	Ignores all <code>*.csv</code> (the crash file alone is 567 MB) and <code>.env</code>

Datasets are **not** committed. Re-download from NYC Open Data, or use `git add -f` for small derived files.

RiskMap build plan v3 · Next.js + Supabase/PostGIS + Mapbox + Open-Meteo · crash data 2,269,187 records (2012-07-01 → 2026-06-11) · FloodNet 2,929 events across 294 sensors · 4,823 complaints scored at severity ≥ 4